DAILY JAVA DEEP DIVES · THE LATEST NEWS, FULLY EXPLAINED

Java – Beautifully Broken

The million-digit Duration, and Shenandoah learns that most objects die young

Issue #11 | Thu, 15 Oct 2026 | ⏱️ ~38 min read | Level 3/5

☕ **Today's two stories**

1. **A Jackson bug lets one JSON request keep a CPU core busy for seconds or minutes, and the newest Spring Boot releases still ship a vulnerable version.** CVE-2026-68497 hides a huge number inside a JSON *string*, which slips past the limits Jackson added in 2023. We trace the request from the parser to a slow JDK constructor, time the slowdown ourselves, read the one-line fix, and explain how to upgrade before Spring's 22 October release train.
2. **JEP 535 will make Shenandoah, the low-pause garbage collector, generational by default in JDK 28.** Its original "collect the whole heap every time" mode is being deprecated. Here's the eight-year path that led here, how a concurrent generational collector actually works, a small experiment showing why it matters, and what changes for anyone running `-XX:+UseShenandoahGC`.
3. **On the radar:** 6 more things that happened, listed at the end.

STORY 1 The million-digit Duration: a Jackson DoS hiding inside a string

LIBRARIES · JACKSON-DATABIND · CVE-2026-68497 (CVSS 7.5) · FIXED 2.18.10 / 2.21.6 / 2.22.2 / 3.1.6 / 3.2.2 · ~15 MIN READ

In 2023, Jackson taught its JSON parser to say no to absurdly long numbers. Send it a number with a million digits and it stops at digit 1,001. It turns out there was a way around that rule: put the million digits inside quotes. A JSON string can be twenty million characters long, and if your class happens to have a field of the right obscure type, Jackson passes that string to a JDK method that slows down with the *square* of its length. We timed it: about eleven seconds of CPU for an 800,000-digit value. One request, one stuck thread.

WHAT HAPPENED

The vulnerability is **CVE-2026-68497**, published as GitHub security advisory *≈ a vulnerability report in GitHub's advisory database, identified by a GHSA-... ID* GHSA-q4xh-88c3-wmh7.[1] It's in jackson-databind *≈ the Jackson module that maps JSON onto Java objects and back ("data binding"). It's the JSON library Spring Boot uses by default*, the part of Jackson almost every Spring application uses through `ObjectMapper`.

The CVE record was published on 11 September 2026 by HeroDevs, acting as the CNA *≈ CVE Numbering Authority: an organisation allowed to assign CVE IDs and publish CVE records*, and is rated 7.5 (High) with the CVSS *≈ Common Vulnerability Scoring System, the standard 0–10 severity score* vector `AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`. In plain

terms, it can be triggered over the network, with no login and no user interaction, and the damage is to **availability**, not to data.[2] It's classed as CWE-400, uncontrolled resource consumption.[1]

According to the advisory, jackson-databind's default deserializers for `javax.xml.datatype.Duration` and `XMLGregorianCalendar` pass the raw JSON string straight to the JDK's `DatatypeFactory`, with no length check. The JDK parses the numeric parts with constructors whose cost grows **quadratically** ≈ *"$O(n^2)$": doubling the input roughly quadruples the work*. Because the digits sit inside a JSON string rather than a number, jackson-core's number-length guard never applies. The advisory says a single 1–5 MB request can tie up a thread for tens of seconds to minutes, **with the default mapper configuration**.[1,2]

Affected and fixed versions, from the CVE record:[2]

Line	Affected	Fixed in
Jackson 2 (`com.fasterxml.jackson.core`)	2.0.0 to 2.18.9, 2.19.0 to 2.21.5, 2.22.0 to 2.22.1	**2.18.10**, **2.21.6**, **2.22.2**
Jackson 3 (`tools.jackson.core`)	3.0.0 to 3.1.5, 3.2.0 to 3.2.1	**3.1.6**, **3.2.2**

The fixes shipped in mid-August, about four weeks before the CVE went public: 3.1.6, 3.2.2 and 2.21.6 on 14 August, 2.18.10 on 15 August and 2.22.2 on 16 August (release tag dates).[6] No 2.19.x or 2.20.x release contains the fix; the newest 2.20 release, 2.20.2, dates from January 2026.[6] HeroDevs lists 2.13, 2.14 and 2.15 as end-of-life lines with no public fix, and offers patched builds of those through its paid support.[3]

This was on our radar twice (issues #1 and #10). It gets a deep dive now because of something we found while checking who's exposed: **the newest Spring Boot releases still manage vulnerable Jackson versions.** More on that below.

THE BACKGROUND YOU NEED

How Jackson reads JSON, in two layers

Jackson is split into layers. **jackson-core** ≈ *Jackson's low-level streaming parser and generator. It turns JSON text into a sequence of tokens and knows nothing about your classes* reads the raw text and produces **tokens** ≈ *the parser's units of meaning: "start of object", "field name", "string value", "number value" and so on*. On top of it, jackson-databind walks those tokens and builds your objects. For each Java type, databind uses a **deserializer** ≈ *a small object that knows how to build one Java type from JSON tokens*.

The split matters for security, because each layer can only defend what it sees. The parser sees text and tokens. Databind sees types.

The limits Jackson added in 2023

Jackson has had denial-of-service bugs before, mostly about deeply nested input. CVE-2020-36518 (fixed in 2.12.7, May 2022) was about recursion in untyped deserialization, and CVE-2022-42003 and CVE-2022-42004 (fixed in 2.13.4.1 and 2.12.7.1, October 2022) were about deeply nested arrays.[7] In April 2023, jackson-core 2.15.0 answered with a general mechanism called **StreamReadConstraints** ≈ *a set of limits that jackson-core*

enforces while parsing: maximum number length, string length, nesting depth and so on. Its release notes list "maximum numeric value length limits" and a limit on the longest textual value as headline changes.[9]

In current 2.18 releases, the defaults are:[10]

- maximum **number** length: **1,000** characters
- maximum **string** length: **20,000,000** characters
- maximum nesting depth: 1,000
- maximum property name length: 50,000

Why limit numbers at all? Because turning a long run of decimal digits into a Java number is expensive. **BigInteger** *≈ the JDK's arbitrary-precision integer class, for numbers too big for a long* and **BigDecimal** *≈ the JDK's arbitrary-precision decimal class, often used for money and exact fractions* build their value from a string with an algorithm whose cost grows with the square of the number of digits.[5] Keep numbers under 1,000 digits and that cost stays negligible.

The weak spot is in the second bullet. A *string* may be twenty million characters long, and the parser has no idea what you'll do with it.

Two XML types that live in the JDK

`javax.xml.datatype.Duration` and `javax.xml.datatype.XMLGregorianCalendar` are not the `java.time` classes you'd normally use. They're the JDK's Java versions of the XML Schema types `xsd:duration` and `xsd:dateTime`. Their text format looks like `P1Y2M3DT4H5M6.5S` (one year, two months, three days, 4 h 5 min 6.5 s) and `2026-10-15T10:00:00.123`. You create them through a **DatatypeFactory** *≈ the JDK factory class (javax.xml.datatype) that parses and creates the XML Schema date, time and duration types*.

You rarely write these types by hand. You *generate* them. **JAXB** *≈ Java Architecture for XML Binding: the standard way to generate Java classes from an XML schema (XSD) and map XML onto them* binds `xsd:dateTime`, `xsd:date` and `xsd:time` to `XMLGregorianCalendar`, and `xsd:duration` to `javax.xml.datatype.Duration`.[13] So these fields turn up in classes generated from XML schemas, for SOAP clients, banking and insurance message formats, and government data standards. When teams later expose the same model classes as a JSON API, Jackson has to read these types from JSON.

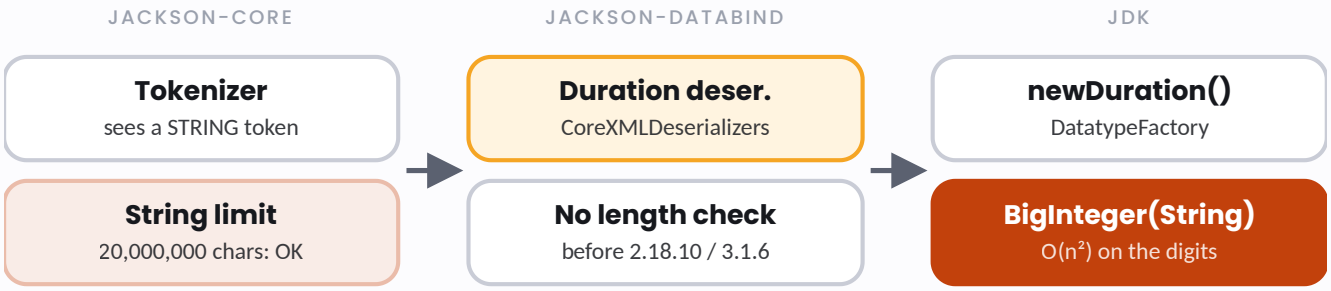
Jackson supports them out of the box. A class in databind called `CoreXMLDeserializers` registers deserializers for `QName`, `XMLGregorianCalendar` and `Duration`. They take a JSON string and hand it to a shared `DatatypeFactory`.[11]

HOW IT WORKS

The path of the malicious value

Here's what happens when a request body binds to a class with a `javax.xml.datatype.Duration` field and the attacker sends `"P"`, then hundreds of thousands of digits, then `"Y"`.

Why the 2023 number limit never fires



The number-length limit (1,000) only applies to JSON number tokens. A number inside quotes is just a string to the parser.

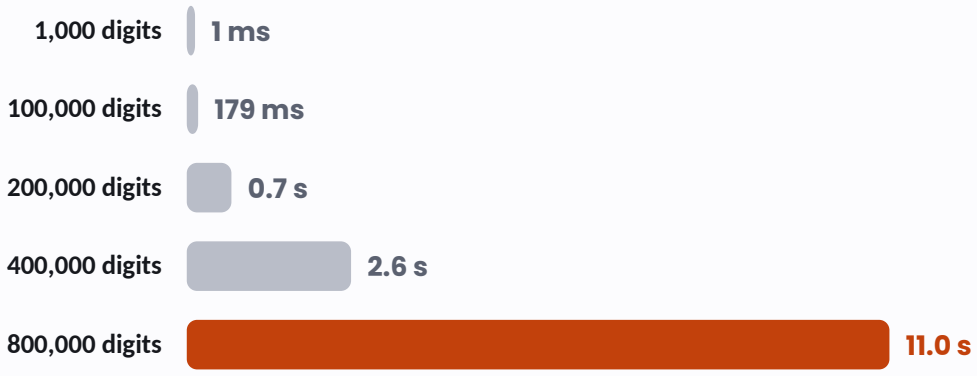
The parser sees an ordinary string well under its 20 million character limit, so it passes. Databind looks up the deserializer for `Duration`, gets the string and calls `_dataTypeFactory.newDuration(value)`.[11] The JDK parses each part of the duration (years, months, days and so on) into a `BigInteger`, and the fractional seconds into a `BigDecimal`. The fix's own code comment describes it this way: it uses "the O(n^2) BigInteger(String)/BigDecimal(String) constructors".[5]

`XMLGregorianCalendar` takes a slightly longer route. Jackson first tries its own date parser. If that fails, it falls back to `newXMLGregorianCalendar(value)`, which parses the fractional seconds into a `BigDecimal` with the same quadratic constructor.[12]

How slow is "quadratic"? We measured the JDK part

We couldn't download Jackson from our sandbox, so we measured the JDK step, which is where the time goes. On OpenJDK 21.0.12 (2 vCPUs), we passed `"P" + n nines + "Y"` straight to `DatatypeFactory.newDuration()`:[14]

Time for one DatatypeFactory.newDuration() call vs number of digits (OpenJDK 21)



Our measurement, single run each, JDK 21.0.12, default DatatypeFactory (Xerces-based, inside the JDK). Each doubling of the digits costs about 4x. A 400,000-digit fractional second in newXMLGregorianCalendar took 2.7 s.

Each doubling of the input made the call about four times slower, which is what quadratic means. 800,000 digits is well under a megabyte of JSON, yet it cost eleven seconds of one core. The advisory's 1–5 MB example is larger still, which is why it talks about minutes.[1] Nothing in the request is malformed, either: 800,000 nines is a valid duration of 999…9 years.

Now look at the scale of a web server. A servlet container or reactive server has a fixed pool of **worker threads** ≈ *the threads a server uses to process requests. When all are busy, new requests queue or get rejected*. If an endpoint accepts this field, a handful of such requests sent in parallel can keep every worker busy. The CVE description makes exactly this point: several concurrent requests can exhaust the server's worker threads.[2]

The fix: treat the whole string as a number

The fix is commit `a99b7e7` (pull request #6127), written by Jackson maintainer PJ Fanning on 29 July 2026 after a report by @waydeshi.[5,7] Its idea is simple. Before calling `DatatypeFactory`, check the **length of the whole string** against the same `maxNumberLength` limit the parser uses for numbers, 1,000 by default.

CoreXMLDeserializers.Std._deserialize()

2.18.9 / 3.1.5

```
case TYPE_DURATION:
  return _dataTypeFactory
      .newDuration(value);

case TYPE_G_CALENDAR:
  Date d;
  try {
    d = _parseDate(value, ctxt);
  } catch (JsonMappingException e) {
    return _dataTypeFactory
      .newXMLGregorianCalendar(value);
  }
```

2.18.10 / 3.1.6

```
case TYPE_DURATION:
  _validateTimestampLength(ctxt, value);
  return _dataTypeFactory
      .newDuration(value);

case TYPE_G_CALENDAR:
  _validateTimestampLength(ctxt, value);
  Date d;
  try {
    d = _parseDate(value, ctxt);
  } catch (JsonMappingException e) { ...
```

Simplified from jackson-databind at the 2.18.10 and 3.1.6 tags. The helper checks value.length() against StreamReadConstraints.getMaxNumberLength() (default 1,000).

Why the whole string and not just the digits? The code comment explains: the string is "never shorter than any single component", so it's a safe upper bound, and "no valid Duration comes anywhere near the (default 1000) limit".[5] A string that's too long is rejected with a **StreamConstraintsException** ≈ *the exception jackson-core throws when input breaks one of the StreamReadConstraints limits* that says the number value length exceeds the maximum allowed. The same change went into the 2.18, 2.21 and 2.22 lines and into Jackson 3's 3.1 and 3.2 lines.[6,12]

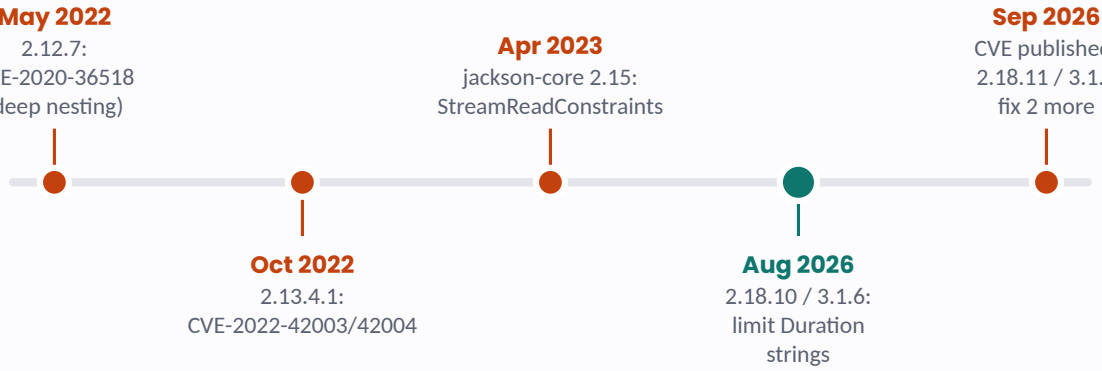
That's the whole fix: one length check, a few lines of comments, and 112 lines of new tests.[5]

Not an isolated bug: Jackson's 2026

CVE-2026-68497 is one of many this year. Counting only the 2.18 line's release notes, 2.18.8 (28 May) to 2.18.11 (20 September) contain **14 fixes labelled with 2026 CVE IDs**, from `@JsonView` bypasses to DNS lookups during `InetAddress` deserialization. The previous CVE-labelled databind fixes in those notes date

from 2022.[7] The pattern of "a number hidden inside a string" also keeps coming back. Jackson 3.1.7 (21 September) applies the number-length limit when a string is coerced to `double` (#6179), and 2.18.11 and 3.1.7 fix two more denial-of-service bugs: CVE-2026-91776 (an unbounded type-id cache) and CVE-2026-91777 (quadratic forward-reference resolution in collection and map deserializers).[8,7] Several of the 2026 fixes are credited in the notes as written "w/ Claude code".[7]

Jackson's defences against expensive input



Sources: jackson-databind VERSION-2.x and VERSION release notes, jackson-core 2.15.0 release notes, CVE record (published 11 Sep 2026).

WHAT IT MEANS FOR YOU

First, check whether you're exposed at all. You're affected if untrusted JSON is bound to a class that has a `javax.xml.datatype.Duration` or `XMLGregorianCalendar` field, directly or in a nested object. HeroDevs notes that exposure only requires binding one of these types from untrusted JSON.[3] A quick search of your code (including generated sources) for `XMLGregorianCalendar` and `javax.xml.datatype.Duration` answers most of it. `java.time.Duration` is a different class, and it isn't the one named in the advisory.

Classes generated from XSDs are the usual suspects.[13] Picture an insurance company with a SOAP-era policy model, generated from a schema, that it now also accepts as JSON on a public REST endpoint. Every `xsd:dateTime` in that model is an `XMLGregorianCalendar` and an entry point.

Then upgrade Jackson, and don't assume Spring Boot has done it for you. We checked the Jackson versions that Spring Boot manages, in the spring-boot repository:[15]

Spring Boot	Released (tag)	Jackson 3	Jackson 2	Status for this CVE
4.1.1	30 Jul 2026	3.1.5	2.21.5	**vulnerable**
4.0.8	30 Jul 2026	3.1.5	2.21.5	**vulnerable**
4.2.0-M2	14 Sep 2026	3.1.6	2.21.6	fixed
4.1.x / 4.0.x branches	6 Oct 2026	3.1.7	2.21.7	fixed, unreleased
3.5.16	25 Jun 2026	–	2.21.4	**vulnerable**

On 6 October, the 4.0.x and 4.1.x branches were upgraded directly from Jackson 3.1.5 to 3.1.7 and to Jackson 2 2.21.7.[15] Spring's new monthly release train, "Patch Thursday", is on the Thursday after the third Monday of

the month, and its first patch release is on **22 October 2026**.[16] That's the earliest the fix is likely to reach you through Spring Boot's dependency management. If you can't wait a week, override the managed version yourself. In Maven, a `dependencyManagement` entry in your own POM wins over the one from the Spring Boot parent. If you import **BOMs** ≈ *"bill of materials" POMs that only pin versions for a family of artifacts, imported into dependencyManagement* instead, import `tools.jackson:jackson-bom:3.1.7` (or `com.fasterxml.jackson:jackson-bom:2.21.7` for Jackson 2) **before** `spring-boot-dependencies`. Then run `mvn dependency:tree` or `./gradlew dependencyInsight --dependency jackson-databind` and check which version actually ends up on the classpath.

Prefer 3.1.7 or 2.21.7 over 3.1.6 or 2.21.6. Both fix this CVE. The newer ones also fix CVE-2026-91776 and CVE-2026-91777.[8]

If you can't upgrade at all, Red Hat's advice is to limit the size and complexity of untrusted JSON input.[4] A request-body size limit at your gateway or server (for example a few hundred kilobytes for an ordinary API) doesn't remove the bug, but it caps the cost of each request. Given our measurement, 100 KB of digits already costs around 0.2 seconds per call. Only a Jackson upgrade puts the cap at 1,000 characters.

🧪 Try it: prove your upgrade worked

Add a small test that feeds your application's own `ObjectMapper` (the one Spring injects, not a fresh one) a 5,000-digit duration. On a fixed Jackson it should fail fast. On a vulnerable one it quietly succeeds, slowly for larger values.

```
record Policy(javax.xml.datatype.Duration term) {}

String json = "{\"term\":\"P" + "9".repeat(5_000) + "Y\"}";
// Fixed: rejected (5,002 chars > maxNumberLength 1,000)
assertThrows(Exception.class,
    () -> mapper.readValue(json, Policy.class));
```

Sketch for JUnit 5 with Jackson 2 or 3. We couldn't run it, because Maven Central is unreachable from our sandbox, so treat the exact exception type as something to check in your own run.

💥 Watch out for

- **The fix inherits your number limit.** The new check uses `StreamReadConstraints.getMaxNumberLength()`. If your team raised that limit (for example to accept very large numbers), these two types now accept strings just as long.[1,2]
- **Jackson 2.19 and 2.20 have no fixed release.** Their branches contain the commit, but no release with it has been tagged. Move to 2.21.6+ or 2.18.10+.[6]
- **End-of-life lines stay vulnerable.** 2.13, 2.14 and 2.15 have no public fix.[3]
- **Transitive copies count.** A fat JAR ≈ *a single JAR that bundles your app together with all its dependencies*, an older SDK that shades Jackson, or a second `ObjectMapper` in a library can bring its own vulnerable copy.
- **Don't confuse the two `Duration` classes.** Search for the `javax.xml.datatype` import, not the word "Duration".

WHAT HAPPENS NEXT

Spring's 22 October train should ship Boot patch releases with Jackson 3.1.7 and 2.21.7, according to the branch state.[15,16] More Jackson advisories are likely. The project has already patched the 2.18 line four times since May, and the "numbers inside strings" pattern is clearly being audited across databind.[7,8]

📖 Words from this story

jackson-databind: the Jackson module that maps JSON onto Java objects and back; Spring Boot's default JSON library.

CNA: CVE Numbering Authority, an organisation allowed to assign CVE IDs.

GitHub security advisory: a vulnerability report in GitHub's advisory database, with a GHSA-… ID.

CVSS: the standard 0–10 severity score for vulnerabilities.

Quadratically (O(n²)): the work grows with the square of the input; doubling the input roughly quadruples the cost.

jackson-core: Jackson's low-level streaming parser and generator.

Tokens: the parser's units, such as "string value" or "number value".

Deserializer: the object that builds one Java type from JSON.

StreamReadConstraints: jackson-core's limits on number length, string length, nesting depth and so on.

BigInteger / BigDecimal: the JDK's arbitrary-precision integer and decimal classes.

DatatypeFactory: the JDK factory that parses XML Schema dates, times and durations.

JAXB: the standard way to generate Java classes from an XML schema and bind XML to them.

Worker threads: the server threads that process requests.

BOM: a "bill of materials" POM that pins versions for a family of artifacts.

Fat JAR: one JAR bundling an app with all its dependencies.

StreamConstraintsException: the exception thrown when input breaks a StreamReadConstraints limit.

STORY 2 Shenandoah learns that most objects die young

JDK 28 · OPENJDK · JEP 535 TARGETED (AUG 2026), IMPLEMENTATION PENDING · ~16 MIN READ

Shenandoah was built to do something almost no other collector could: move objects around while your program keeps running, so that pauses stay short whether the heap is 200 MB or 200 GB. But it had a blind spot. Every collection cycle walked the *entire* heap, including the gigabytes of objects that had been alive for hours and would stay alive for hours more. JEP 535 makes the fix, a generational Shenandoah, the default in JDK 28. It took eight years and one false start to get here.

WHAT HAPPENED

JEP 535, "Shenandoah GC: Generational Mode by Default", is Targeted to JDK 28.[18] It became Proposed to Target in late July, with the review due to end on 3 August 2026, and InfoQ reported it as Targeted in its roundup for the week of 3 August.[17,18] In InfoQ's words, the JEP "proposes to designate the Shenandoah Garbage Collector to generational mode by default", and "the non-generational mode will be deprecated with the intent to remove it in a future release".[17]

JDK 28 is due in March 2027. The JDK 28 feature list now includes JEP 535 alongside value objects (JEP 401), strict field initialization (JEP 539), the macOS/x64 deprecation (JEP 541), the JSON API (JEP 540) and PEM encodings (JEP 542).[19]

The change hasn't landed in the code yet. In the OpenJDK master branch as of 15 October (commit `44647eb`), `ShenandoahGCMode` still defaults to `satb`, the original single-generation mode, named after **SATB** ≈ *snapshot-at-the-beginning: a concurrent marking technique that treats everything reachable when marking starts as live, and uses a barrier to record references the application overwrites while marking runs* marking.[20] (A note on sources: the JEP page itself, openjdk.org/jeps/535, wouldn't load from our sandbox, so the JEP's exact wording comes to us via InfoQ. The technical detail below comes from the earlier JEPs and from the HotSpot source code.)

THE BACKGROUND YOU NEED

Garbage collection in one paragraph

Your objects live on the **heap** ≈ *the memory area where the JVM allocates objects*. A **garbage collector** ≈ *the part of the JVM that finds objects your program can no longer reach and reclaims their memory* periodically works out which objects are still reachable, by following references from the **roots** ≈ *the starting points for reachability: local variables on thread stacks, static fields and a few JVM internals*, and frees the rest. Many collectors also **compact** ≈ *move live objects together so free memory forms large contiguous blocks* the heap. The hard part is doing this while your program, which GC people call the **mutator** ≈ *the application threads, which keep changing ("mutating") the object graph while the GC works*, keeps changing the very graph being traced. The simplest answer is to stop the world: a **GC pause** ≈ *a period when all application threads are halted so the collector can work safely*.

What made Shenandoah different

Red Hat's Christine Flood and Roman Kennke designed Shenandoah to shrink those pauses. JEP 189 describes its goal as pause times that don't grow with heap size: the same at 200 MB or 200 GB.[25] It got there by making the expensive part, **evacuation** ≈ *copying live objects out of a region so the whole region can be freed*, happen **concurrently** with the running application. That's hard, because a thread might read an object at the moment the GC is moving it. JEP 189 describes the trick as an extra indirection pointer per object, so GC threads can compact while Java threads run, and says the only pause it really needs is to scan thread stacks to find the roots. The price is extra CPU work and memory.[25] Today's Shenandoah implements this with a **load reference barrier** ≈ *a small piece of code the JIT inserts wherever a reference is loaded from the heap. If the object is being moved, it returns the new copy*.[33]

Shenandoah arrived in JDK 12 as an experimental collector (JEP 189) and became a product feature in JDK 15 (JEP 379), the same release where ZGC dropped its experimental label.[25,26,27] One practical detail surprises many people: **Oracle's own JDK builds don't include Shenandoah.** Red Hat's Andrew Haley documented this in 2019 for Oracle's open-source JDK 12 builds, which answered `-XX:+UseShenandoahGC` with "not supported".[28] Oracle's GC tuning guide lists Serial, Parallel, G1 and ZGC, and no Shenandoah.[29] You get it from other builds: Red Hat's (from JDK 8 onwards, according to Haley), Amazon's Corretto, which shipped the generational preview, or the Ubuntu OpenJDK 21 package we used for the experiment below.[28,30]

The blind spot: no generations

Most collectors in the JVM are **generational** ≈ *they split the heap into a young area for new objects and an old area for long-lived ones, and collect the young area much more often*. They rely on the **generational hypothesis** ≈ *the observation that most objects die young: they become garbage soon after they're created*. If that's true, collecting just the young area finds a lot of garbage for little work, because the long-lived objects in the old area don't need to be traced every time.

Shenandoah, in its original mode, has no generations. Each cycle marks every live object in the heap. JEP 404 explains what that costs: compared with G1, CMS and Parallel, single-generation Shenandoah "needs more heap headroom" and works harder to reclaim unreachable objects. It works well only within "limited operational envelopes" of heap occupancy and allocation rate.[33] When the application allocates faster than a whole-heap concurrent cycle can finish, Shenandoah falls back to a **degenerated GC** ≈ *Shenandoah's fallback when a concurrent cycle can't keep up: it finishes the cycle with application threads stopped*, or in the worst case a **full GC** ≈ *a complete stop-the-world collection and compaction of the whole heap*, and the low-pause promise breaks.

Our small experiment

To see the blind spot, we ran a toy program on JDK 21 (whose Shenandoah has only the single-generation mode). It keeps about 430 MB of objects alive for the whole run and, for 20 seconds, allocates short-lived 1 KB arrays as fast as it can, in a 2 GB heap on a 2-vCPU machine. Same program, two collectors:[38]

- **G1** ran 114 young collections. Typical pauses were 2–3 ms, the longest was 70 ms, and the total was 0.67 s. It never needed to trace the 430 MB of old objects.

- **Shenandoah** ran 97 full cycles. Every pause stayed under 0.2 ms, which is its whole point. But each cycle re-marked the entire live heap, about 115 ms of concurrent marking per cycle and **11.1 seconds of marking in total**, all on threads competing with the application for two CPUs.

Same 20-second toy workload: GB the application allocated (more = more work done)



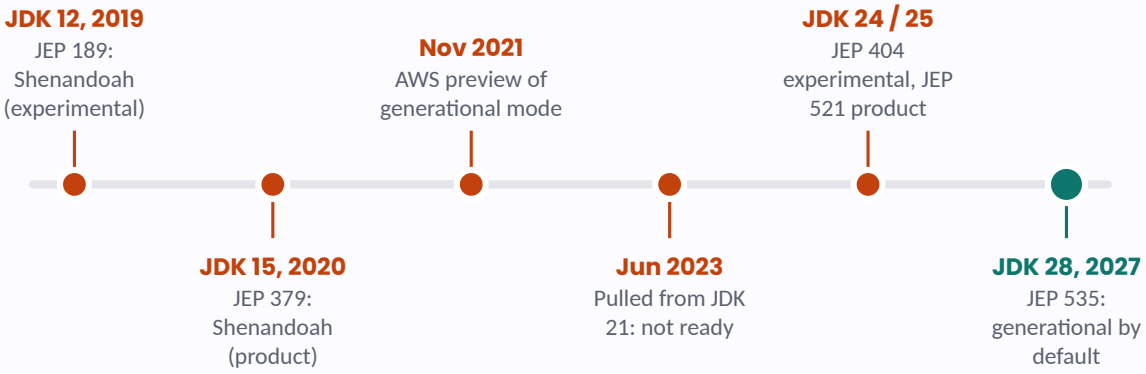
Our measurement, single run, OpenJDK 21.0.12 (Ubuntu build), 2 vCPU, -Xms2g -Xmx2g, ~430 MB long-lived data. Allocation volume is a rough proxy for application progress. Shenandoah: 97 cycles, max pause 0.2 ms; G1: 114 young pauses, max 70 ms. Illustrative only; JDK 21 has no generational Shenandoah, so we couldn't measure it.

That's the trade, made concrete: tiny pauses, but about a third less application work in this run, because the collector kept re-reading data that never changed. Generational mode is meant to keep the tiny pauses and skip most of that re-reading.

HOW IT WORKS

The long road to a generational Shenandoah

Shenandoah, from experiment to generational default



Sources: JEP 189, JEP 379, Oracle's Java 15 announcement, AWS Developer Tools Blog (30 Nov 2021), InfoWorld (9 Jun 2023), JEP 404, OpenJDK 'JEPs in JDK 25', InfoQ.

The generational work came largely from Amazon. On 30 November 2021, the Amazon Corretto team (Mike Cook, David Alvarez, Paul Hohensee, Kelvin Nilsen and William Kemper), working with Red Hat, released a preview for Corretto 17. It was aimed at applications allocating more than 4 GB/s or with more than 60% of the heap live.[30] Amazon claimed that some workloads matched single-generation Shenandoah's response times with about a third of the heap, with p99 pauses under 10 ms and less than 5% throughput cost. Those are vendor claims.[30] An AWS executive called it "the first of its kind a concurrent generational garbage collector for the JVM".[31]

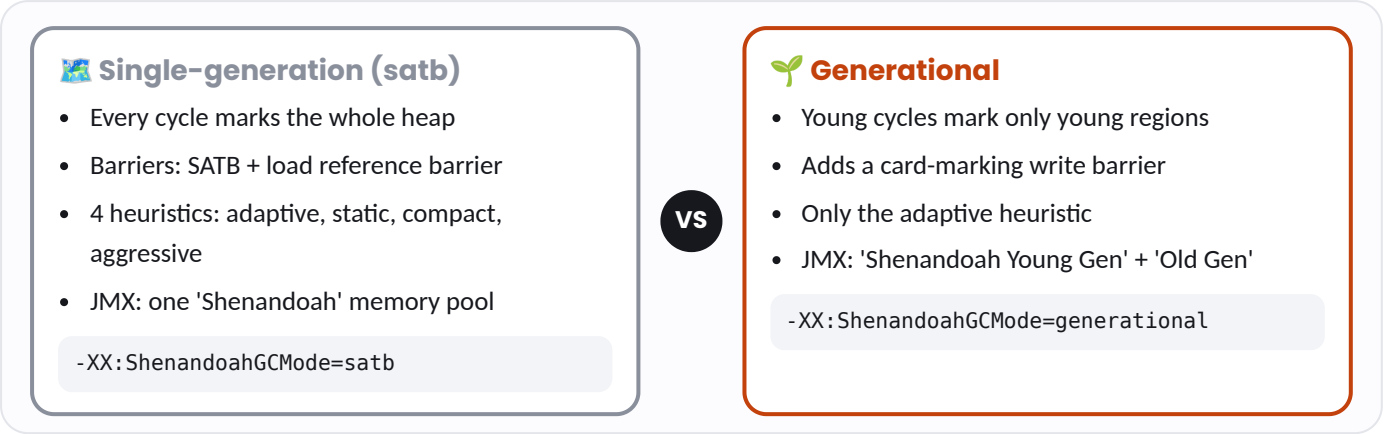
The road to mainline was bumpy. Generational Shenandoah was set for JDK 21 as an experimental feature, but in June 2023 it was removed for "a lack of readiness".[32] It finally arrived as experimental in JDK 24 (JEP

404), and became a product feature in JDK 25 (JEP 521), where it no longer needs `-XX:+UnlockExperimentalVMOptions`.[33,34,35] JEP 521 explicitly did *not* change the default.[34] That's JEP 535's job.

ZGC has walked this path already. Generational ZGC (JEP 439) became ZGC's default in JDK 23 (JEP 474), and the old mode was removed in JDK 24 (JEP 490).[35] JEP 474's stated goals were to signal that future development focuses on the generational mode and to "reduce the maintenance cost of supporting two modes".[36,37] JEP 535 follows the same pattern. The difference is timing: ZGC's removal came one release after the default switch. JEP 535 only promises removal "in a future release".[17]

What a generational Shenandoah does differently

JEP 404 describes the design.[33] The heap is still divided into **regions** *≈ fixed-size chunks of the heap that the collector manages individually*, but each region is now either free, young or old.



Young collections. Most cycles collect only the young generation. They use Shenandoah's usual concurrent machinery (marking, then concurrent evacuation through the load reference barrier), so pauses stay short.[33]

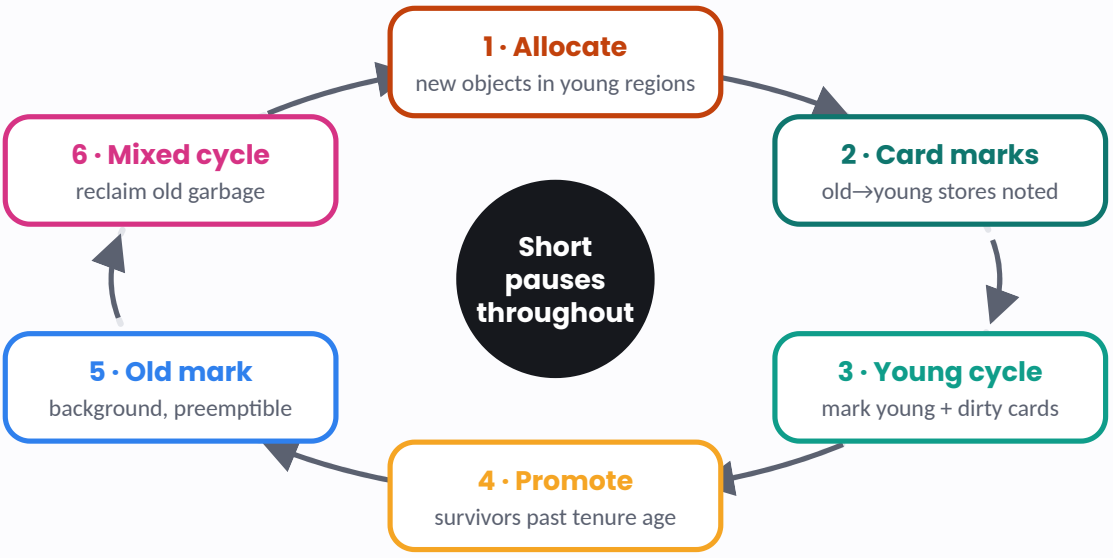
The remembered set. There's a catch. If a young collection traces only young objects, how does it know that some old object still points at a young one? Generational collectors keep a **remembered set** *≈ a record of where old objects hold references into the young generation, so a young collection can treat those references as extra roots*. Shenandoah's is a **card table** *≈ a byte array in which each entry ("card") covers a small slice of the heap. A "dirty" card means that slice may contain a reference worth checking*. Whenever your code stores a reference into an object, a **write barrier** *≈ a few instructions the JIT adds after every reference store; here, it marks the card for that address as dirty* marks the card. In the source, this is `ShenandoahCardBarrier`, described as "true when ShenandoahGCMode is generational, false otherwise".[20] JEP 404 says the card marking code was borrowed from the Parallel and CMS collectors, plus new code that lets remembered-set scanning run concurrently with your program.[33]

Old collections and mixed cycles. The old generation is marked in the background, and young collections are allowed to interrupt that marking (`ShenandoahAllowOldMarkingPreemption`).[33,20] Once old marking is complete, later cycles become **mixed collections** *≈ cycles that evacuate young regions plus some old regions with the most garbage*. G1 users will recognise the idea, except here it runs concurrently.[33]

Promotion. Objects that survive enough young cycles are **promoted** *≈ moved (tenured) from the young generation to the old one*. The tenuring age isn't fixed. Shenandoah keeps a census of age cohorts and their death rates, and adapts the age between a floor of 1 and a ceiling of 15. A cohort whose mortality falls below 10% is treated as

long-lived.[20] It also keeps 8 regions in reserve so that promotions can't use up the space needed to compact the old generation.[20]

A generational Shenandoah heap in steady state



What the JEP 535 switch will change, in flags

Today (JDK 25 to 27), generational mode is a product option you have to ask for. JEP 521 says the command line is `-XX:+UseShenandoahGC -XX:ShenandoahGCMode=generational`, and leaving `-XX:+UnlockExperimentalVMOptions` in is not an error.[34] After JEP 535, plain `-XX:+UseShenandoahGC` should give you the generational mode. Keeping the old behaviour will mean asking for the existing `satb` mode, which the JEP deprecates.[17]

Choosing a Shenandoah mode

JDK 25–27

```
# default: single-generation
java -XX:+UseShenandoahGC App

# opt in to generational
java -XX:+UseShenandoahGC \
  -XX:ShenandoahGCMode=generational \
  App
```

JDK 28 (JEP 535, expected)

```
# default: generational
java -XX:+UseShenandoahGC App

# opt out (deprecated mode)
java -XX:+UseShenandoahGC \
  -XX:ShenandoahGCMode=satb \
  App
```

Left from JEP 521. Right is our reading of JEP 535's summary as reported by InfoQ; the implementation and its exact warning messages haven't landed yet (master, 15 Oct 2026).

WHAT IT MEANS FOR YOU

Most Java developers: nothing changes. G1 remains the JVM's default collector, and JEP 535 only changes what happens once you've chosen Shenandoah. If you run Oracle JDK, you can't select Shenandoah anyway.[29]

Shenandoah users: test generational mode now, on JDK 25. You don't have to wait for JDK 28. JDK 25, the current LTS, already ships generational Shenandoah as a product feature.[34,35] Run your load test with `-XX:ShenandoahGCMode=generational` and compare pause times, throughput, CPU and memory with your current settings. Workloads like the ones Amazon targeted, with high allocation rates or a mostly-full heap, have the most to gain.[30] When JDK 28 arrives, you'll already know the result.

Check your configuration flags. Three things in the current HotSpot code are worth knowing before the default flips:[21,22]

1. **Only one heuristic.** A **heuristic** ≈ *in Shenandoah, the policy that decides when to start a cycle and how much to collect* is chosen with `-XX:ShenandoahGCHeuristics`. Generational mode supports only `adaptive`. If you set `compact`, `static` or `aggressive`, the JVM logs a warning that it's "Ignoring -XX:ShenandoahGCHeuristics input" because "generational shenandoah only supports adaptive heuristics" and uses `adaptive` instead. A team that picked `compact` to keep memory low would quietly lose that behaviour after the switch.
2. **Different ergonomic defaults.** Generational mode sets `GCTimeRatio` to 70, and doesn't apply the `TLABAllocationWeight` value of 90 that single-generation mode sets, keeping the JVM's standard default of 35.
3. **Platform support.** Generational mode runs only on x86-64, AArch64, PPC64 and RISC-V 64. On any other platform, the JVM exits with "Shenandoah Generational GC is not supported on this platform". How JEP 535 handles those platforms isn't something we could verify without the JEP text.

Update your dashboards. Monitoring tools read memory pools through **JMX** ≈ *Java Management Extensions, the JVM's built-in monitoring API used by tools like JConsole, Micrometer and APM agents*. Single-generation Shenandoah exposes one heap pool named `Shenandoah` (we checked on JDK 21). Generational mode registers two pools, `Shenandoah Young Gen` and `Shenandoah Old Gen`.[23] An alert on "pool `Shenandoah` above 90%" will silently stop matching anything. GC log lines will also look different.

Remember the costs. JEP 404's risk list still applies: scanning the remembered set can lengthen pauses, the card-marking barrier adds a little overhead to every reference store, and the sizing and promotion heuristics may need tuning for unusual workloads.[33] JEP 474 warned ZGC users that "some workloads are non-generational by nature" and may run slightly slower in the generational mode.[36] Expect the same here.

💥 Watch out for

- **Not merged yet.** JEP 535 is Targeted, but the default in master is still `satb` (15 Oct). Details like warning texts may change before JDK 28.[20]
- **"Deprecated" isn't "removed".** In JDK 28 you'll still be able to choose the single-generation mode. Plan to move off it, as ZGC removed its old mode one release after deprecating it.[35]
- **Heuristic flags are ignored, not rejected.** Read your startup logs after upgrading.[21]
- **Not in Oracle JDK.** If your ops team "switched to Shenandoah" on Oracle JDK, check what's actually running.[28]
- **Vendor numbers are claims.** The 2021 AWS figures (one third of the heap, under 5% throughput cost) predate five years of changes. Measure your own workload.[30]

WHAT HAPPENS NEXT

The implementation still has to be reviewed and integrated before JDK 28's feature freeze. Once it lands, the JDK 28 early-access builds will let you test the new default directly. Removing the single-generation mode will need a separate JEP in a later release, as it did for ZGC.[17,35] Meanwhile Shenandoah keeps getting a steady stream of fixes and performance work in master, including better balancing of remembered-set scanning across GC threads, a part only the generational mode uses, and memory footprint reductions.[24]

📖 Words from this story

SATB: snapshot-at-the-beginning, Shenandoah's concurrent marking technique and the name of its single-generation mode.

Heap: the memory area where the JVM allocates objects.

Garbage collector: the JVM component that finds unreachable objects and reclaims their memory.

Roots: the starting points for reachability, such as thread stacks and static fields.

Compact: move live objects together so free memory is contiguous.

Mutator: GC jargon for the application threads.

GC pause: a period when application threads are stopped for the collector.

Evacuation: copying live objects out of a region so the region can be freed.

Load reference barrier: JIT-inserted code on reference loads that returns an object's new copy if it's being moved.

Generational: splitting the heap into young and old areas and collecting the young one more often.

Generational hypothesis: most objects die young.

Degenerated GC: Shenandoah's fallback that finishes a cycle with the application stopped.

Full GC: a complete stop-the-world collection of the whole heap.

Regions: fixed-size heap chunks the collector manages individually.

Remembered set: a record of old-to-young references used as extra roots in young collections.

Card table: a byte array where each entry covers a slice of the heap and can be marked dirty.

Write barrier: JIT-inserted code after reference stores; here, it dirties the card.

Mixed collections: cycles that evacuate young regions plus the most garbage-filled old regions.

Promoted (tenured): moved from the young to the old generation after surviving enough cycles.

Heuristic: Shenandoah's policy for when to start a cycle and how much to collect.

JMX: Java Management Extensions, the JVM's built-in monitoring API.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **Oracle's 20 October Critical Patch Update is the first JDK 21 update under the OTN licence:** Oracle JDK 21 updates were free under NFTC only through September 2026, so check which licence your next JDK 21 download falls under (issue #1 has the options).[39]
- **"Everyday JSON in Java" at Devoxx Belgium:** Alan Bateman and Viktor Klang present the incubating JDK JSON API as a low-friction option for small JSON jobs, not a replacement for libraries like Jackson.[40]
- **JobRunr 9 lands on Spring Initializr:** the background-job library's new major version can now be selected when generating a Spring Boot project.[41]
- **Jetty 12.1.14 and 12.0.40, Vert.x 5.2.1, 5.1.10 and 4.5.35, and Apache Camel 4.18.5:** a round of maintenance releases for widely used server and integration libraries.[42]
- **Azul Zing builds of OpenJDK, release 26.08:** Azul's commercial JVM with its own JIT and pauseless collector published its August release notes.[42]
- **Kotlin Toolchain 0.13:** JetBrains' new Kotlin project setup tooling, announced for Kotlin Multiplatform apps.[42]

---

1. **CVE-2026-68497: jackson-databind Duration XMLGregorianCalendar Unbounded Number Parse DoS**, GitLab Advisory Database. GitHub advisory: **GHSA-q4xh-88c3-wmh7** (didn't load from our sandbox).
2. **CVE-2026-68497**, OpenCVE (CVE record by HeroDevs as CNA, published 11 Sep 2026; NVD last modified 18 Sep 2026).
3. **CVE-2026-68497**, HeroDevs vulnerability directory.
4. **CVE-2026-68497**, Red Hat Customer Portal.
5. **Commit a99b7e7: Add StreamReadConstraints number len constraint to GregorianCalendar and Duration (#6127)**, FasterXML/jackson-databind, 29 Jul 2026 (5 files, +141 lines).
6. Release tags in **FasterXML/jackson-databind**, read with git on 15 Oct 2026: 3.1.6, 3.2.2 and 2.21.6 (14 Aug), 2.18.10 (15 Aug), 2.22.2 (16 Aug), 2.18.11 (20 Sep), 2.21.7, 2.22.3, 3.1.7, 3.2.3 (21 Sep), 2.20.2 (19 Jan 2026). Commit a99b7e7 is contained in branches 2.18 to 2.22 and 3.1/3.2.
7. **jackson-databind release notes, VERSION-2.x**, at tag 2.18.11.
8. **jackson-databind 3.x release notes, VERSION**, at tag 3.1.7.
9. **jackson-core release notes, VERSION-2.x**, 2.15.0 (23 Apr 2023) entries #815, #863, #943.
10. **StreamReadConstraints.java**, jackson-core 2.18.11 (default limits, `validateIntegerLength`, `validateFPLength`).
11. **CoreXMLDeserializers.java**, jackson-databind 2.18.9 (before the fix; same code as the parent of commit a99b7e7).
12. **CoreXMLDeserializers.java** at 2.18.10, and the **Jackson 3 version** at 3.1.6; helper `_validateTimestampLength` in `StdDeserializer` (@since 2.18.10).
13. **Binding XML Schemas (Table 17-1)**, The Java EE 5 Tutorial, Oracle.
14. Our measurement, 15 Oct 2026: `DatatypeFactory.newInstance().newDuration("P" + "9".repeat(n) + "Y")` and `newXMLGregorianCalendar` with a long fractional second, OpenJDK 21.0.12.1 (Ubuntu build), 2 vCPU sandbox, one run per size after a warm-up call. Jackson itself was not run (Maven Central is unreachable from our sandbox).
15. spring-boot `gradle.properties` (`jacksonVersion`, `jackson2Version`) at tags **v4.1.1**, **v4.0.8** and **v4.2.0-M2**, and on branches 4.1.x/4.0.x; **v3.5.16**; upgrade commits **2b8ff29**, **35a4635** and **1a6afb4** (6 Oct 2026). Read with git on 15 Oct 2026.
16. **Releasing Spring for Modern Challenges**, spring.io, 21 Sep 2026.
17. **Java News Roundup (week of July 27th, 2026)**, InfoQ, 3 Aug 2026.
18. **Java News Roundup: Shenandoah GC, TeamCity CVE, A2A Java SDK, Camel, Gradle, GlassFish, Groovy**, InfoQ, 10 Aug 2026. JEP page: **openjdk.org/jeps/535** (didn't load from our sandbox).
19. **Java 28**, javaalmanac.io (JEP list; release 2027-03).

20. **shenandoah_globals.hpp**, openjdk/jdk master at commit 44647eb (15 Oct 2026): `ShenandoahGCMode` default `"satb"`, `ShenandoahCardBarrier`, tenuring, `ShenandoahOldCompactionReserve` and `ShenandoahAllowOldMarkingPreemption` flags.
21. **shenandoahArguments.cpp**, openjdk/jdk master (heuristics warning, `TLABAllocationWeight`).
22. **shenandoahGenerationalMode.cpp**, openjdk/jdk master (platform check, card barrier, `GCTimeRatio` 70).
23. **shenandoahMemoryPool.cpp** and **shenandoahGenerationalHeap.cpp**, openjdk/jdk master; pool name on JDK 21 checked with `ManagementFactory.getMemoryPoolMXBeans()`.
24. **openjdk/jdk commit history**, e.g. 8391086 "Shenandoah: Remembered set scan needs to balance queues better" and 8393152 "Shenandoah: Further optimize HdrSeq memory footprint", read with git on 15 Oct 2026.
25. **JEP 189: Shenandoah: A Low-Pause-Time Garbage Collector (Experimental)**, OpenJDK (delivered in JDK 12).
26. **JEP 379: Shenandoah: A Low-Pause-Time Garbage Collector (Production)**, OpenJDK.
27. **The Arrival of Java 15**, Oracle Java blog.
28. **Not all OpenJDK 12 builds include Shenandoah: Here's why**, Andrew Haley, Red Hat Developer, 19 Apr 2019.
29. **Available Collectors**, HotSpot Virtual Machine Garbage Collection Tuning Guide, Java SE 17, Oracle.
30. **Announcing preview release for the generational mode to the Shenandoah GC**, AWS Developer Tools Blog, 30 Nov 2021.
31. **Generational Shenandoah Offers Java a Faster Way to Collect Garbage**, Joab Jackson, The New Stack, 14 Dec 2021.
32. **Java 21 to drop generational Shenandoah GC feature**, Paul Krill, InfoWorld, 9 Jun 2023.
33. **JEP 404: Generational Shenandoah (Experimental)**, OpenJDK (delivered in JDK 24).
34. **JEP 521: Generational Shenandoah**, OpenJDK.
35. **JEPs in JDK 25 integrated since JDK 21**, OpenJDK (JEP 474 in JDK 23, JEP 490 in JDK 24, JEP 521 in JDK 25).
36. **JEP 474: ZGC: Generational Mode by Default**, OpenJDK (delivered in JDK 23).
37. **Java ZGC update (JEP 474)**, InfoQ, 3 May 2024.
38. Our measurement, 15 Oct 2026: a toy program holding 3,000,000 `long[16]` arrays (~430 MB) and allocating 1 KB `byte[]`s into a 4,096-slot ring for 20 s; OpenJDK 21.0.12.1 (Ubuntu build), 2 vCPU, `-Xms2g -Xmx2g -Xlog:gc`, one run per collector. Process CPU: G1 20.7 s, Shenandoah 25.8 s.
39. **JDK 21 approaches end of permissive license**, Oracle Java blog, 14 Aug 2026.
40. **Everyday JSON in Java**, Inside.java, 14 Oct 2026.
41. **This Week in Spring - October 13th, 2026**, spring.io.
42. **Java Weekly, Issue 667**, Baeldung, 9 Oct 2026.